

Homework 5 - Report

Hande Eryilmaz, 21902678

Use of GenAI Tools

This homework assignment was completed with the assistance of **Claude Sonnet 4.5**. The following describes how the tool was used throughout the assignment:

Claude was used to help implement the MATLAB code for all parts of this assignment, assist in structuring and drafting the written commentary and observations sections of this report. All written content was reviewed, understood, and verified by the student. When errors occurred during code execution, the error messages were shared with Claude to identify and correct the issues. The student remains fully responsible for all submitted code, results, and written content. All MATLAB code was executed and verified locally by the student. All numerical outputs, figures, and conclusions were checked for correctness and consistency. The student is able to explain and justify all code, results, and interpretations presented in this report.

Part 1 - Display the Data

Objective and Methodology

The objective of this section is to load the multicoil MRI dataset, display the coil images and their corresponding k-space spectrums, and compute the Sum-of-Squares (SoS) reference image that will be used as the gold standard throughout the homework.

The dataset contains fully sampled images `im` of size $256 \times 256 \times 8 \times N_x \times N_y \times N_c$ acquired using an 8-channel receive coil array, along with a random sampling mask of size 256×256 . For each coil, the centered 2D FFT is computed to obtain the k-space data. The SoS combination is then computed as:

$$\hat{m}SoS = \sqrt{\sum_c 1^{N_c} |m_c|^2}$$

The SoS image is normalized by its 95th percentile pixel intensity using `prctile(sos(:), 95)` to define the reference normalization scale. The variable `ref_val` is defined as the maximum pixel intensity of the normalized SoS image and is used throughout the homework for consistent display windowing and PSNR/SSIM computation.

Observations and Comments

Coil images: The 8 coil images show spatially varying sensitivity patterns characteristic of a phased array receive coil. Each coil is brightest in the region of the brain closest to its physical location — coils in the upper row show stronger sensitivity in the superior brain regions, while coils in the lower row are brighter in the inferior regions. The individual coil images have lower SNR than the combined SoS image due to the limited spatial coverage of each coil element.

K-space spectrums: The log-magnitude k-space images show the characteristic bright center (low-frequency, high-energy content) with energy decaying toward the periphery. The k-space appears somewhat grainy and noisy in the outer regions — this is expected because the original data had size 220×160 and was zero-padded to 256×256 in k-space. The zero-padded outer regions contain near-zero values that appear as noise in the logarithmic display.

SoS reference image: The SoS combination produces a high-quality brain image with good anatomical detail. The cortical gyri, sulci, and overall brain structure are clearly visible. The SoS image shows more uniform intensity distribution across the brain compared to individual coil images, since each brain region is covered by at least one coil with good sensitivity. This normalized SoS image serves as the reference `ref_img` for all subsequent PSNR and SSIM computations.

Normalization: The 95th percentile normalization value is `p95 = 0.6778`, and after normalization the maximum pixel intensity is `ref_val = 2.1610`. This means a small number of pixels (the brightest 5%) exceed the 95th percentile threshold, consistent with bright boundary pixels typical in SoS images.

Code Snippet and Command Window Output

Code:

```
load('/Users/handeeryilmaz/Downloads/medikal/homework_475_5/multicoil-random.mat');
[Nx, Ny, Nc] = size(im);

% Compute k-space
M = zeros(Nx, Ny, Nc);
for c = 1:Nc
    M(:,:,c) = fftshift(fft2(ifftshift(im(:,:,c))));
end

% Display coil images
figure; montage(abs(im), 'DisplayRange', [], 'Size', [2 4]);
colormap gray;
title('Part 1 - Coil Images (Magnitude)');

% Display k-space (log scale)
M_log = log(abs(M) + 1);
figure; montage(M_log, 'DisplayRange', [], 'Size', [2 4]);
colormap gray;
title('Part 1 - K-space Spectrums (Log Magnitude)');

% SoS combination
sos = sqrt(sum(abs(im).^2, 3));

% 95th percentile normalization
p95 = prctile(sos(:), 95);
sos_ref = sos / p95;
ref_val = max(sos_ref(:));

fprintf('p95: %.4f\n', p95);
fprintf('ref_val: %.4f\n', ref_val);

% Display SoS reference
figure;
imshow(sos_ref, [0 ref_val]);
colormap gray; colorbar;
title('Part 1 - SoS Reference Image (95th pct normalized)');
```

Command Window Output:

```
Image size: 256 x 256 x 8
Mask size: 256 x 256
p95: 0.6778
ref_val: 2.1610
Part 1 complete. ref_val = 2.1610
```

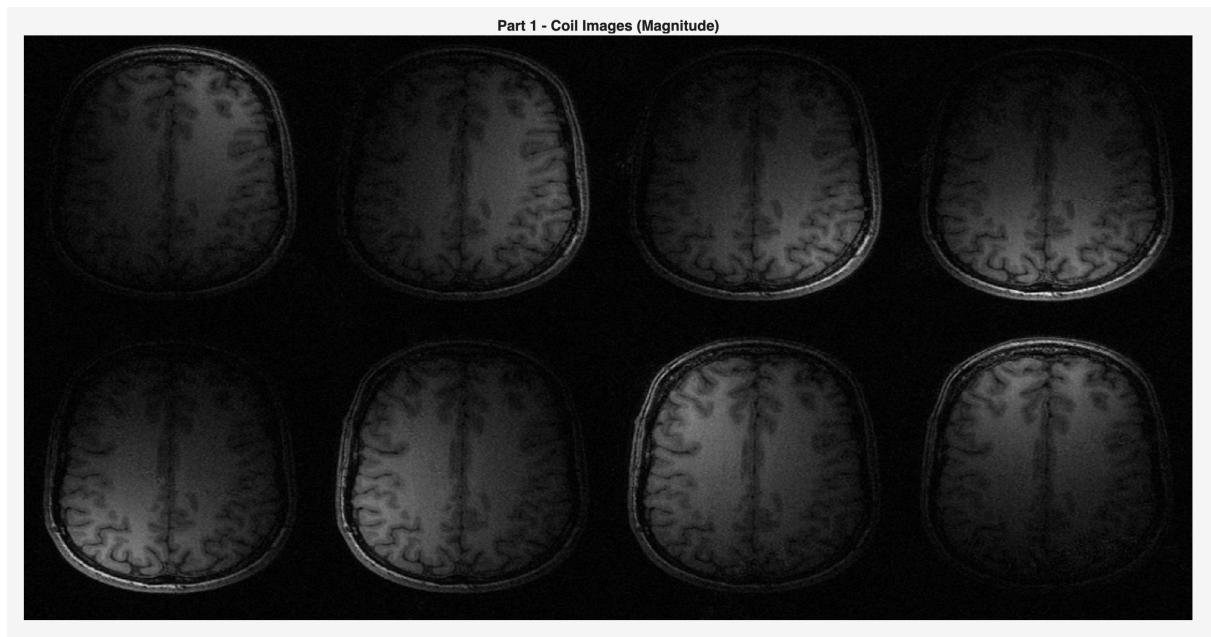


Figure 1: Part 1 - Coil Images (Magnitude) for all 8 coils

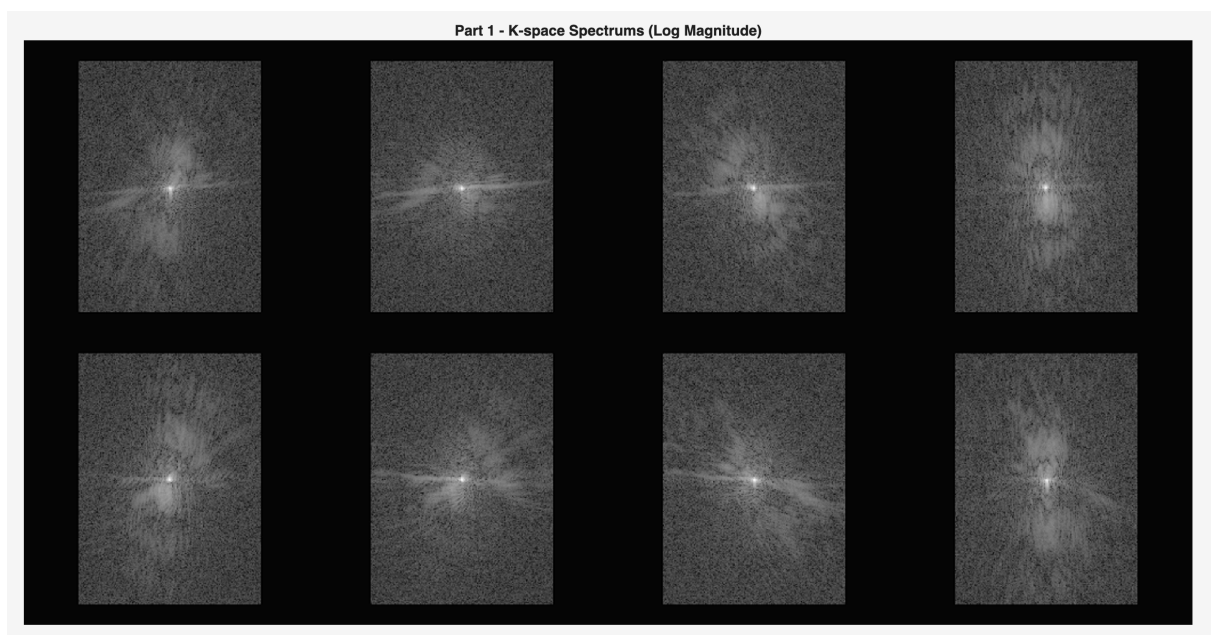


Figure 2: Part 1 - K-space Spectrums (Log Magnitude) for all 8 coils

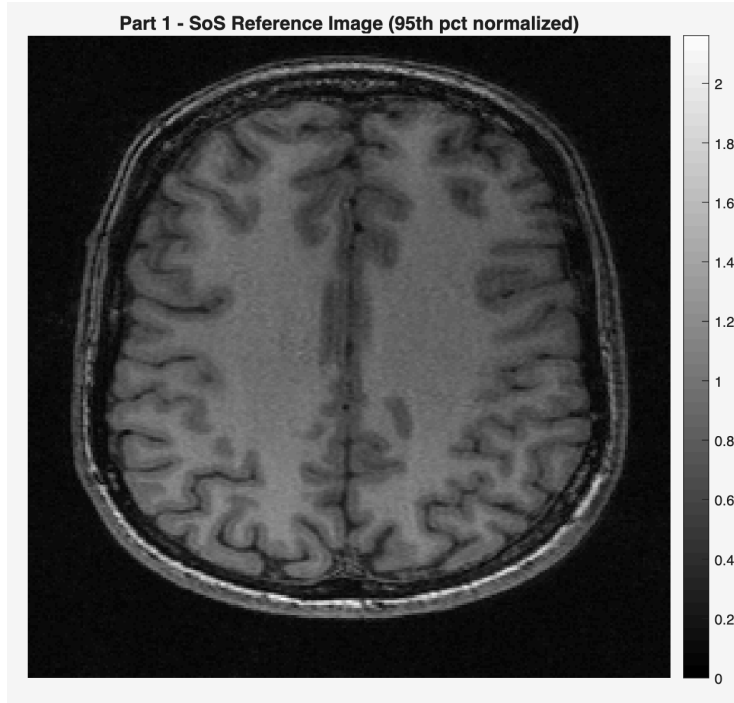


Figure 3: Part 1 - SoS Reference Image (95th percentile normalized)

Part 2 - Sampling Mask and Random Undersampled Images

Objective and Methodology

The objective of this section is to display the random sampling mask, determine the acceleration factor, and implement the `randundersample(im, mask)` function that generates undersampled k-space data and zero-fill reconstructed images.

The `randundersample` function performs the following steps for each coil:

1. Compute the centered 2D FFT of the coil image to obtain full k-space data
2. Apply element-wise multiplication with the binary mask to generate undersampled k-space `Mu`
3. Apply centered inverse 2D FFT of `Mu` to obtain zero-fill reconstructed images `imu`

Both `imu` and `Mu` have size $256 \times 256 \times 8$. The acceleration factor is computed as:

$$R = \frac{N_{total}}{N_{sampled}} = \frac{256 \times 256}{N_{sampled}}$$

The SoS combination of the zero-fill images is normalized by its own 95th percentile. PSNR and SSIM are computed by normalizing both the undersampled SoS image and the reference SoS image by `ref_val`.

Observations and Comments

Sampling mask: The mask shows the characteristic variable-density random sampling pattern used in compressed sensing MRI. The central 32×32 region is fully sampled (bright white square) — this is the calibration region needed for SPIRiT kernel estimation. Outside the calibration region, the sampling density decreases progressively from center to periphery, with more points sampled near the center and fewer at the outer k-space edges. This variable-density design is intentional — the center of k-space contains the most energy and must be well-sampled to avoid severe image degradation.

Acceleration factor: The mask samples 13,219 out of 65,536 total k-space points, giving an acceleration factor of:

$$R = \frac{65536}{13219} \approx 4.96 \times$$

This is approximately 5x acceleration — significantly undersampled.

Undersampled k-space: The undersampled k-space spectrums show the random sampling pattern clearly — many points are zero (black) with randomly scattered acquired points (white dots). The central calibration region is fully sampled and appears as a bright solid square at the center of each k-space image. Compared to uniform undersampling in HW4 where entire rows/columns were missing, here individual random points are missing throughout k-space.

Zero-fill coil images: The zero-fill reconstructed coil images show the brain anatomy is still clearly recognizable — the overall brain structure and coil sensitivity patterns are preserved from Part 1. However, the images show a spatially incoherent noise-like texture overlaid on the brain anatomy. This is the key characteristic of random undersampling aliasing.

SoS zero-fill image: The SoS combination of zero-fill images shows good overall brain anatomy with incoherent noise-like artifacts spread across the entire image. The brain structure is recognizable but there is a visible noisy texture everywhere.

Error image: The error image reveals that the errors are spatially spread across the entire brain in a noise-like incoherent pattern. The errors are strongest at the brain boundaries and in high-frequency regions (cortical folds). The noise-like appearance of the error confirms the incoherent nature of random undersampling aliasing.

Quantitative metrics:

- PSNR = **28.95 dB** — significantly better than HW4 SENSE results (~17 dB) and comparable to GRAPPA (~22 dB) even without any reconstruction!
- SSIM = **0.8276** — high structural similarity, confirming that brain structure is well preserved

Key comparison with HW4 uniform undersampling: In HW4 with uniform undersampling (e.g., Rx=1, Ry=2), aliasing appeared as discrete coherent ghosting — clear shifted copies of the brain overlapping with the original. With random undersampling here, the aliasing is **incoherent** — it appears as noise-like texture spread uniformly across the image rather than discrete ghosts. This is the fundamental advantage of random undersampling for compressed sensing:

- **Uniform undersampling** → coherent aliasing (discrete ghosts) → hard to remove without sensitivity maps
- **Random undersampling** → incoherent aliasing (noise-like) → can be removed by sparsity-promoting regularization (l1 wavelet, SPIRiT) since the true image is sparse in wavelet domain but the aliasing is not

The incoherent aliasing acts as additive noise that can be suppressed by exploiting the sparsity of MRI images in transform domains, which is the basis of compressed sensing and l1-SPIRiT reconstruction developed in the subsequent parts.

Code Snippet and Command Window Output

Code:

```
function [imu, Mu] = randundersample(im, mask)
    [Nx, Ny, Nc] = size(im);
    Mu = zeros(Nx, Ny, Nc);
    imu = zeros(Nx, Ny, Nc);
    for c = 1:Nc
        M_full = fftshift(fft2(ifftshift(im(:,:,c))));
        Mu(:,:,c) = M_full .* mask;
        imu(:,:,c) = fftshift(ifft2(ifftshift(Mu(:,:,c))));
    end
end

% Acceleration factor
n_sampled = sum(mask(:));
```

```

n_total = numel(mask);
R        = n_total / n_sampled;

% Apply function
[imu, Mu] = randundersample(im, mask);

% SoS and metrics
sos_u = sqrt(sum(abs(imu).^2, 3));
sos_u_n = sos_u / prctile(sos_u(:), 95);
psnr_u = psnr(sos_u_n/ref_val, sos_ref/ref_val);
ssim_u = ssim(sos_u_n/ref_val, sos_ref/ref_val);

```

Command Window Output:

```

Total k-space points: 65536
Sampled k-space points: 13219
Acceleration factor R: 4.9577
imu size: 256 x 256 x 8
Mu size: 256 x 256 x 8
Zero-fill Undersampled SoS:
PSNR: 28.9548 dB
SSIM: 0.8276

```

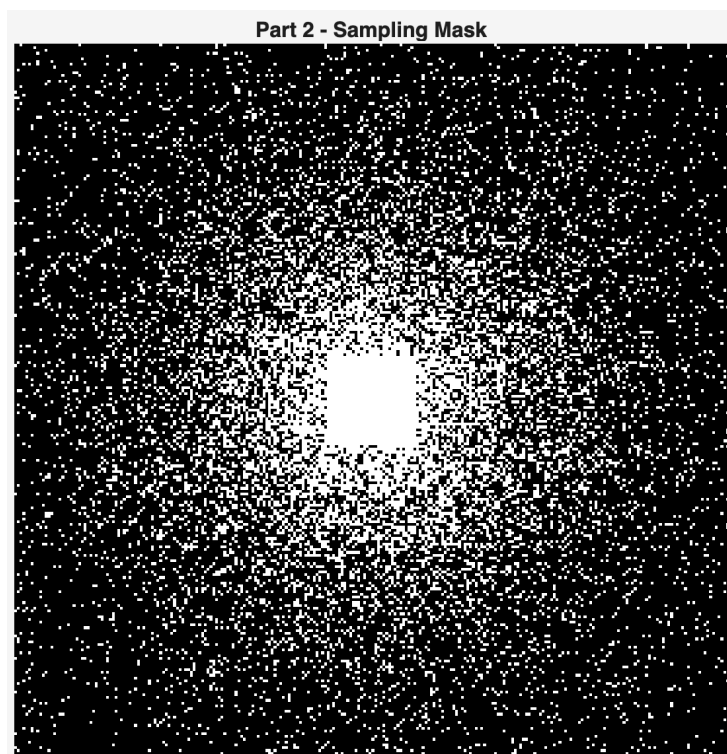


Figure 4: Part 2 - Sampling Mask

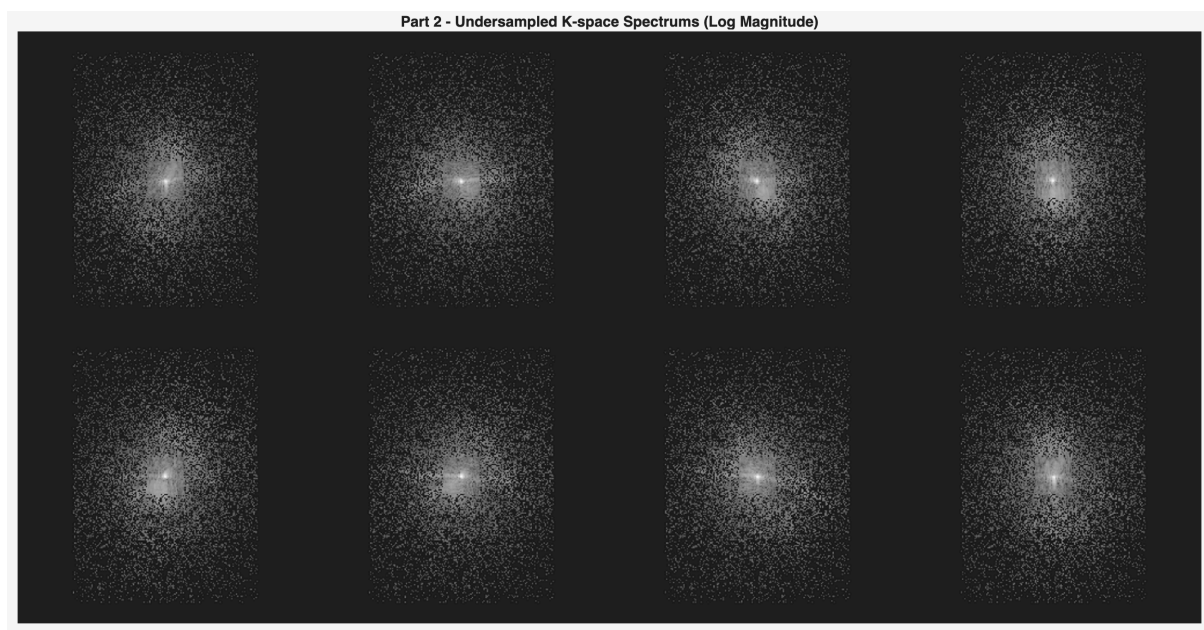


Figure 5: Part 2 - Undersampled K-space Spectrums (Log Magnitude)

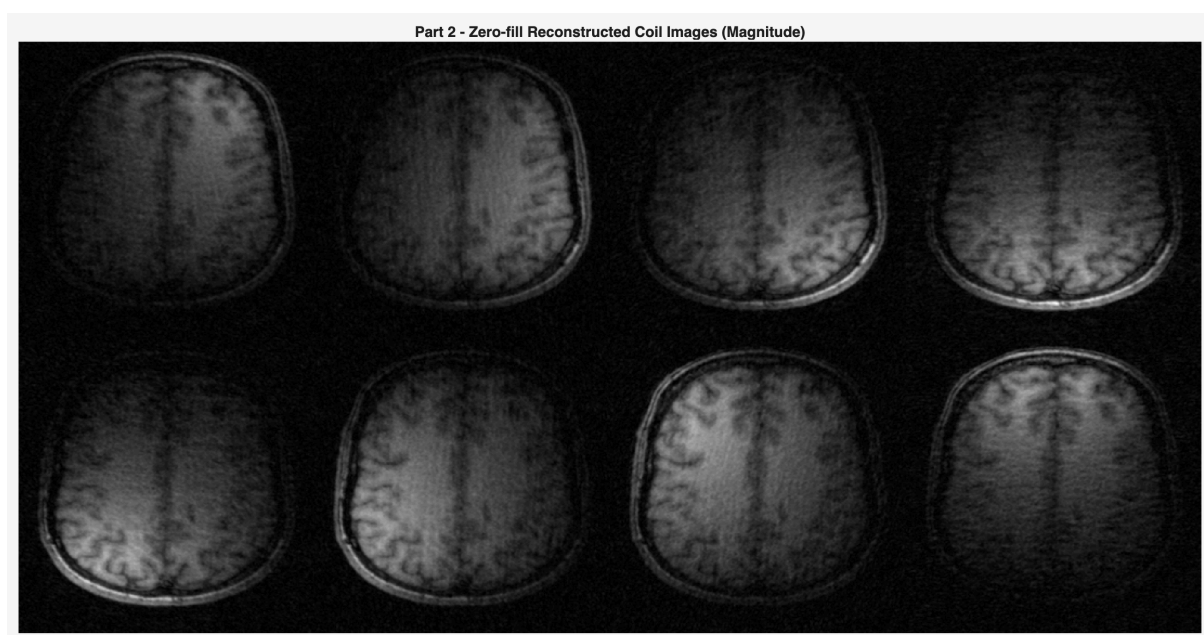


Figure 6: Part 2 - Zero-fill Reconstructed Coil Images (Magnitude)

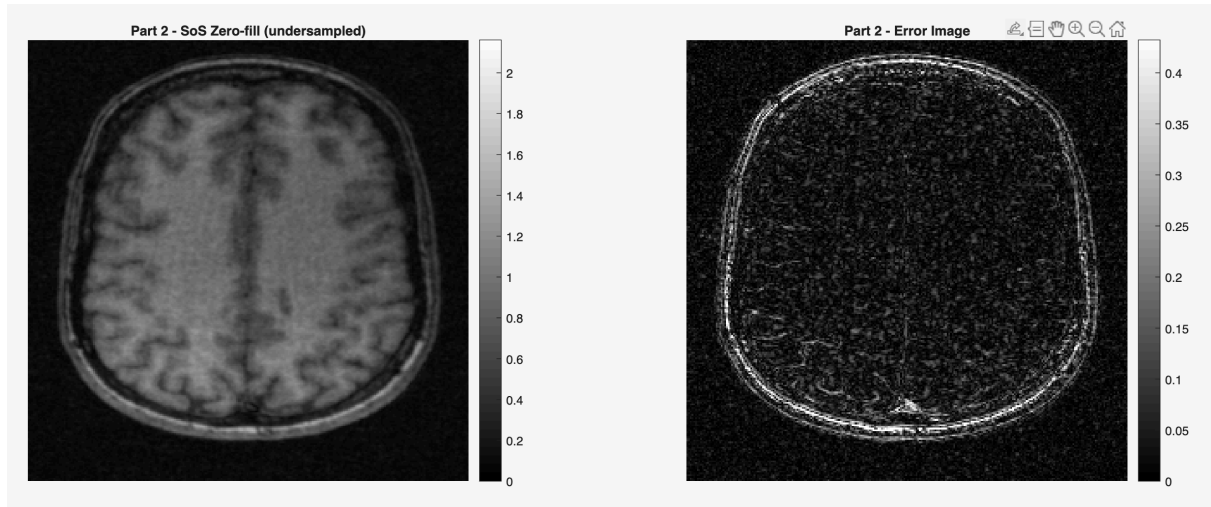


Figure 7: Part 2 - SoS Zero-fill and Error Image

Part 3 - l1 Regularization in Wavelet Domain

Objective and Methodology

The objective of this section is to implement the `llwavelet(imr, beta)` function that applies soft thresholding in the wavelet domain to promote sparsity, and to evaluate its effect on the randomly undersampled coil images from Part 2.

The `llwavelet` function performs the following steps:

1. Apply forward Daubechies-4 wavelet transform with 4 decomposition levels: `coeffW = wv * imr`
2. Find the maximum wavelet coefficient magnitude: $|w|_{max} = \max |w|$
3. Compute threshold: $\tau = \beta \cdot |w|_{max}$
4. Apply soft thresholding to each complex wavelet coefficient:

$$w_{th}(w) = \frac{w}{|w|} \max(0, |w| - \beta |w|_{max})$$

1. Apply inverse wavelet transform: `imth = wv' * coeffW_th`

The function is applied independently to each complex-valued coil image. SoS combination is then performed on the thresholded coil images and results are evaluated for three values of β : 10^{-3} , 10^{-2} , and 5×10^{-2} .

Observations and Comments

Wavelet coefficients before thresholding: The wavelet coefficient images show the characteristic multiresolution structure of the wavelet transform. The energy is highly concentrated in the top-left corner of each image — this corresponds to the coarsest scale approximation coefficients which capture the overall low-frequency brain structure. The remaining regions (detail coefficients at finer scales) appear very dark in log scale, indicating that MRI images are indeed sparse in the wavelet domain — most detail coefficients have very small magnitude. The incoherent aliasing from random undersampling spreads across all wavelet scales, making it distinguishable from the sparse signal content.

Effect of β on reconstruction quality:

β	PSNR (dB)	SSIM	Effect
10^{-3} (small)	28.69	0.826	Very mild thresholding — nearly identical to zero-fill
10^{-2} (medium)	25.96	0.648	Moderate thresholding — visible smoothing

β	PSNR (dB)	SSIM	Effect
5×10^{-2} (large)	22.88	0.442	Heavy thresholding — severe over-smoothing

Table 1: Part 3 - l1 Wavelet Regularization PSNR and SSIM for 3 β values

Key observation — PSNR decreases monotonically with β : Unlike the expected behavior where an intermediate β would optimize PSNR, all three tested values produce PSNR equal to or lower than the zero-fill baseline (28.95 dB). This indicates that for standalone l1 wavelet regularization without data consistency enforcement, even the smallest tested threshold ($\beta=10^{-3}$) begins to remove true signal content along with the aliasing. The optimal β for standalone wavelet thresholding is smaller than 10^{-3} for this dataset.

$\beta=10^{-3}$ (small threshold): The reconstruction looks visually very similar to the zero-fill image from Part 2. The incoherent aliasing noise is marginally reduced but most aliasing remains. The error image still shows the same spatially spread noise-like pattern as the zero-fill case. PSNR=28.69 dB is very close to but slightly below the zero-fill PSNR of 28.95 dB — confirming that very little thresholding is applied.

$\beta=10^{-2}$ (medium threshold): The reconstruction shows noticeably smoother appearance with reduced noise-like texture. However, fine anatomical details (cortical folds, sulci) begin to be lost. The error image shows structured errors at high-frequency regions indicating that true image detail is being removed. PSNR drops to 25.96 dB.

$\beta=5 \times 10^{-2}$ (large threshold): The reconstruction shows severe over-smoothing — the brain appears blurry with significant loss of cortical detail. Structured banding artifacts appear, consistent with aggressive thresholding of detail wavelet coefficients. PSNR drops to 22.88 dB, significantly worse than zero-fill. The error image shows large structured errors throughout the brain interior.

Conclusion: Standalone l1 wavelet thresholding without data consistency is ineffective for compressed sensing reconstruction — it cannot distinguish aliasing from true image content in the wavelet domain. The key insight is that l1 wavelet regularization must be combined with data consistency enforcement (as in l1-SPIRiT in Parts 8-9) to be effective. Data consistency ensures that the acquired k-space samples are not distorted, while wavelet regularization suppresses the aliasing in the unsampled locations. Without data consistency, thresholding indiscriminately removes both aliasing and true signal, leading to degraded reconstruction.

Code Snippet and Command Window Output

Code:

```
function imth = llwavelet(imr, beta)
    wv      = Wavelet('Daubechies', 4, 4);
    coeffW  = wv * imr;
    w_max   = max(abs(coeffW(:)));
    thresh  = beta * w_max;
    abs_cW  = abs(coeffW);
    coeffW_th = coeffW ./ (abs_cW + eps) .* max(0, abs_cW - thresh);
    imth = wv' * coeffW_th;
end

% Apply for 3 beta values
beta_vals = [1e-3, 1e-2, 5e-2];
for b = 1:3
    imu_th = zeros(256,256,8);
    for c = 1:8
        imu_th(:,:,c) = llwavelet(imu(:,:,c), beta_vals(b));
    end
    sos_th  = sqrt(sum(abs(imu_th).^2,3));
    sos_th_n = sos_th / prctile(sos_th(:),95);
    psnr_val = psnr(sos_th_n/ref_val, sos_ref/ref_val);
```

```

ssim_val = ssim(sos_th_n/ref_val, sos_ref/ref_val);
end

```

Command Window Output:

```

Wavelet operator created successfully.
beta=1e-03 | PSNR=28.6854 dB | SSIM=0.8258
beta=1e-02 | PSNR=25.9604 dB | SSIM=0.6476
beta=5e-02 | PSNR=22.8782 dB | SSIM=0.4417

```

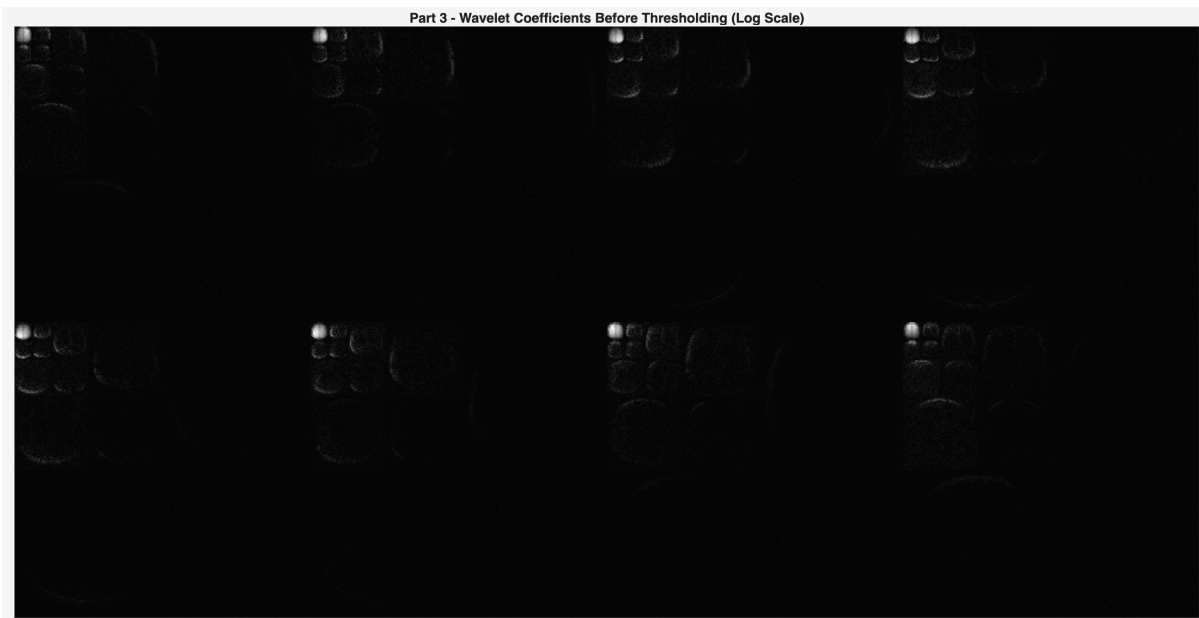


Figure 8: Part 3 - Wavelet Coefficients Before Thresholding (Log Scale)

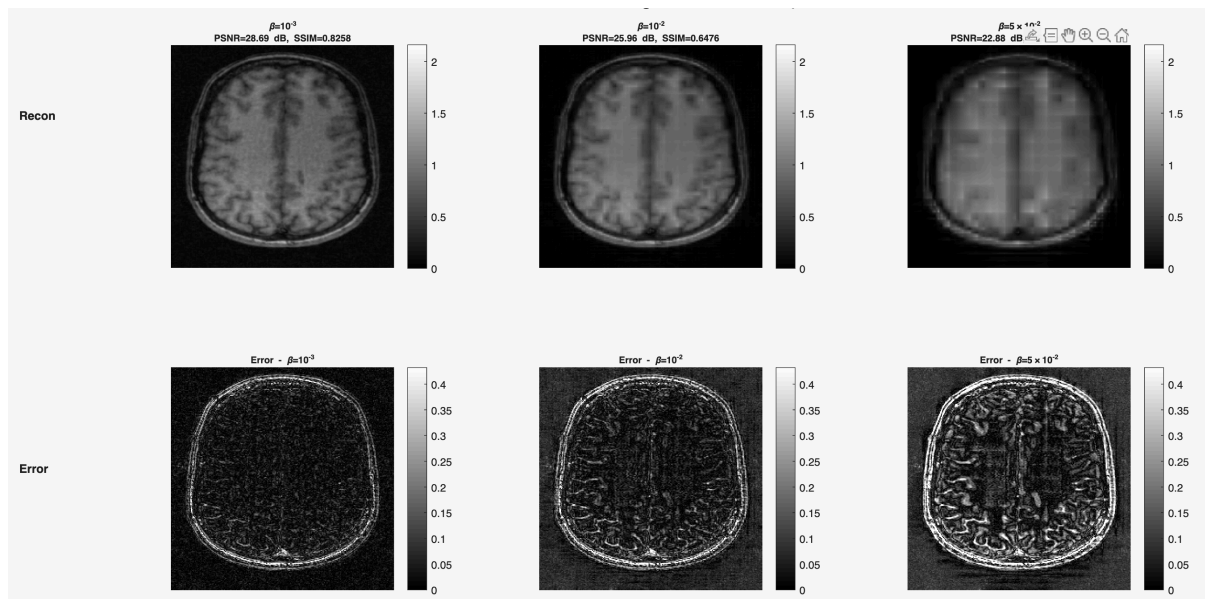


Figure 9: Part 3 - 11 Wavelet Regularization for 3 β Values

Part 4 - SPIRiT Kernel Weights

Objective and Methodology

The objective of this section is to implement the `calibrateSpirit(Mc, lambda)` function that estimates the SPIRiT convolution kernel from the 32×32 central calibration region of k-space.

SPIRiT (Self-consistent Parallel Imaging Reconstruction) is based on the self-consistency principle: in a fully sampled multi-coil dataset, every k-space point can be predicted as a linear combination of its neighbors across all coils. This relationship is captured by the SPIRiT kernel, which is learned from the fully sampled calibration region and then applied to reconstruct the entire k-space.

For a 3×3 kernel with $N_c=8$ coils, the source structure at each target location is:

- **Target coil c:** 8 neighborhood samples (all 3×3 points except the center itself)
- **All other coils:** All 9 points from the 3×3 neighborhood

This gives a total of $8 + 9 \times (N_c - 1) = 8 + 63 = 71 = 9 \times N_c - 1$ source points per target, making the kernel size **71×8**.

The calibration procedure builds the source matrix **A** ($n_{\text{patches}} \times 71$) and target matrix **B** ($n_{\text{patches}} \times 8$) by sliding a 3×3 window over the 32×32 calibration region. The number of calibration patches is $(32 - 3 + 1)^2 = 900$. The kernel is then estimated by solving the regularized least squares problem:

$$\text{kernel} = (\mathbf{A}^H \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^H \mathbf{B}$$

Since the SPIRiT kernel is **sample geometry independent** — it captures the coil-to-coil relationships inherent to the physical coil array — the same kernel applies for all k-space locations and all acceleration levels.

Observations and Comments

Kernel dimensions verified:

- Calibration matrix **A**: 900×71 ✓ (900 patches, 71 source points each)
- Target matrix **B**: 900×8 ✓ (900 patches, 8 target coils)
- Kernel: **71×8** ✓ (71 source points predicting 8 target coils)

The system is well overdetermined: 900 equations for 71 unknowns per column (ratio ≈ 12.7:1), ensuring a stable and accurate kernel estimate.

Kernel magnitude image: The kernel magnitude shows a sparse pattern with distinct bright spots arranged in a diagonal-like structure against a dark background. Each column corresponds to one target coil, and each row corresponds to one source point (from the 71 source positions across all coils). The bright spots indicate source point and coil combinations that contribute most strongly to predicting each target coil.

The diagonal arrangement of bright spots reflects the self-consistency structure of SPIRiT: each coil is most strongly predicted by its own immediate neighbors (the 8 surrounding k-space points of the same coil), with weaker but non-negligible contributions from neighboring coils. This is physically meaningful — the k-space of a given coil is most correlated with its own nearby k-space samples, and adjacent coils in the array provide secondary correlation due to overlapping sensitivity profiles.

The sparse nature of the kernel (most entries near zero, few dominant entries) confirms that the SPIRiT self-consistency relationship is well-localized: only a few source points per coil dominate the prediction. This sparsity is consistent with the smooth spatial variation of coil sensitivity profiles, which translates to smooth and localized k-space interpolation kernels.

λ=0 choice: The regularization parameter $\lambda=0$ is used for display as specified by the homework. Since the calibration system is significantly overdetermined ($900 \gg 71$), the solution is stable without regularization. A non-zero λ will be selected by trial and error in Parts 5-9 to optimize reconstruction quality.

Comparison with GRAPPA kernel: Both SPIRiT and GRAPPA use calibration-based kernel estimation. The key difference is that GRAPPA learns separate kernels to predict only the **missing** k-space lines from **acquired** neighbors, making it specific to the undersampling pattern. SPIRiT instead learns a **universal self-consistency** kernel that applies to every k-space location regardless of whether it was acquired or estimated — this makes SPIRiT applicable to arbitrary sampling patterns including the random undersampling used in this homework.

Code Snippet and Command Window Output

Code:

```

function kernel = calibrateSpirit(Mc, lambda)
    [calibx, caliby, Nc] = size(Mc);
    kSize = 3; kHalf = 1; kk = 9;
    n_src    = kk * Nc - 1;    % 71
    n_patches = (calibx-kSize+1) * (caliby-kSize+1); % 900
    center_in_patch = kHalf * kSize + (kHalf + 1); % = 5

    A = zeros(n_patches, n_src);
    B = zeros(n_patches, Nc);
    patch_idx = 1;

    for px = 1 : calibx - kSize + 1
        for py = 1 : caliby - kSize + 1
            src_vec = zeros(1, n_src);
            col = 1;
            for c = 1:Nc
                patch_c = Mc(px:px+kSize-1, py:py+kSize-1, c);
                p_vec    = patch_c(:)';
                if c == 1
                    keep = true(1, kk); keep(center_in_patch) = false;
                    src_vec(col:col+kk-2) = p_vec(keep);
                    col = col + kk - 1;
                else
                    src_vec(col:col+kk-1) = p_vec;
                    col = col + kk;
                end
            end
            cx_p = px + kHalf; cy_p = py + kHalf;
            B(patch_idx, :) = squeeze(Mc(cx_p, cy_p, :))';
            A(patch_idx, :) = src_vec;
            patch_idx = patch_idx + 1;
        end
    end
    kernel = (A'*A + lambda*eye(n_src)) \ (A'*B);
end

% Extract calibration and compute kernel
Mc    = extractCalib(Mu, 32);
kernel = calibrateSpirit(Mc, 0);

```

Command Window Output:

```

Calibration region Mc: 32 x 32 x 8
Calibrating SPIRiT kernel (lambda=0e+00)...
  n_src (source points per target): 71
  n_patches:                        900
  Calibration matrix A: 900 x 71
  Target matrix B:      900 x 8
  Kernel size: 71 x 8
Expected kernel size: 71 x 8
Actual   kernel size: 71 x 8
Part 4 complete.

```

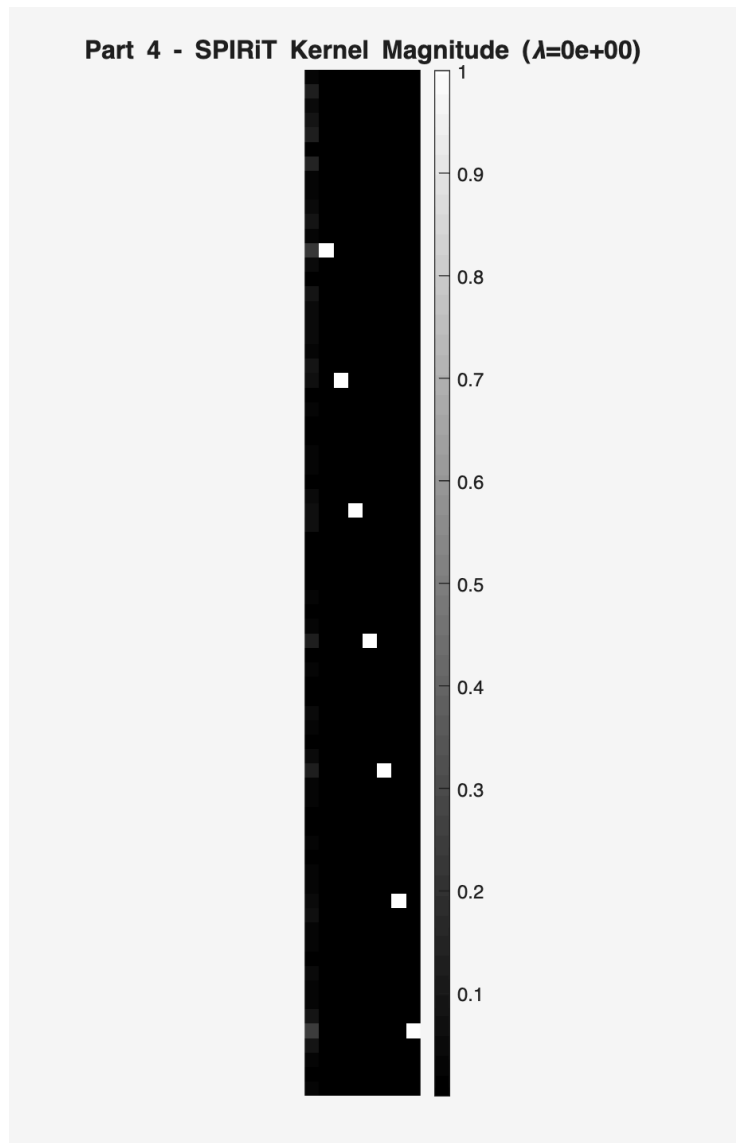


Figure 10: Part 4 - SPIRiT Kernel Magnitude ($\lambda = 0$)

Part 5 - SPIRiT Reconstruction (One Iteration, No Data Consistency)

Objective and Methodology

The objective of this section is to implement the `spirit(Mu, kernel)` function that applies one iteration of SPIRiT reconstruction by convolving the k-space data with the learned kernel. Crucially, **no data consistency** is enforced — all k-space points including the acquired ones are replaced by kernel-predicted values.

The `spirit` function performs the following steps:

1. Pad k-space by $k_{\text{Half}}=1$ on each side with zeros for border handling
2. For each k-space location (x,y) , extract the $3 \times 3 \times 8$ neighborhood patch
3. Build the 71-element source vector using the same structure as calibration: 8 neighbors from coil 1, all 9 points from coils 2-8
4. Apply the kernel to predict all 8 coils simultaneously: `predicted = src_vec × kernel` → 1×8
5. Store predicted values in output k-space `Mr`
6. Apply centered inverse 2D FFT to obtain reconstructed coil images `imr`

Both `imr` and `Mr` are $256 \times 256 \times 8$. The key distinction from GRAPPA is that SPIRiT replaces **every** k-space point (both acquired and unacquired) with its kernel prediction, enforcing global self-consistency. Data consistency is handled separately in Part 6.

A lambda sweep over $[0, 10^{-4}, 10^{-3}, 10^{-2}]$ was performed to select the optimal regularization parameter.

Observations and Comments

Lambda sensitivity: Similar to the GRAPPA findings in HW4, the SPIRiT reconstruction is completely insensitive to λ across the tested range — PSNR=28.4702 dB and SSIM=0.8223 for all four tested values. This is because the calibration system is significantly overdetermined (900 patches $\gg$ 71 unknowns), making the regularization term negligible. $\lambda=10^{-3}$ is selected as a numerically stable choice.

Reconstruction quality (1 iter, no DC):

- PSNR = **28.47 dB** — slightly below zero-fill baseline (28.95 dB)
- SSIM = **0.8223** — slightly below zero-fill (0.8276)

Why PSNR is slightly lower than zero-fill: Without data consistency, SPIRiT replaces the acquired k-space samples with kernel-predicted values. Since the kernel is estimated from a small 32×32 calibration region and applied with a 3×3 support, the predictions at acquired locations are not perfect — they introduce small errors that slightly degrade the reconstruction compared to simply keeping the acquired data. This demonstrates that data consistency is essential for SPIRiT to be effective.

Coil images: The SPIRiT coil images show similar quality to the zero-fill images from Part 2, with the same incoherent noise-like aliasing pattern. The brain anatomy and coil sensitivity patterns are well preserved. The SPIRiT kernel has partially smoothed the random aliasing pattern but without data consistency cannot fully resolve it.

K-space: The output k-space shows a more uniformly filled appearance compared to the sparse undersampled k-space from Part 2. The kernel has interpolated values at previously unsampled locations, but the predicted values at acquired locations have also been modified, losing the true measured data.

SoS image and error: The SoS reconstruction looks similar to the zero-fill result visually, with the same noise-like texture. The error image shows spatially incoherent errors spread across the entire brain, confirming that one SPIRiT iteration without data consistency provides minimal improvement over zero-fill.

Conclusion: A single SPIRiT iteration without data consistency is insufficient for high-quality reconstruction. The self-consistency constraint alone cannot resolve the aliasing — it must be combined with data consistency (replacing acquired samples back after each kernel application) and iterated multiple times, as implemented in Parts 6 and 7.

Code Snippet and Command Window Output

Code:

```
function [imr, Mr] = spirit(Mu, kernel)
    [Nx, Ny, Nc] = size(Mu);
    kSize = 3; kHalf = 1; kk = 9;
    center_in_patch = 5;
    Mu_pad = padarray(Mu, [kHalf, kHalf, 0], 0, 'both');
    Mr = zeros(Nx, Ny, Nc);

    for x = 1:Nx
        for y = 1:Ny
            patch = Mu_pad(x:x+kSize-1, y:y+kSize-1, :);
            src_vec = zeros(1, kk*Nc-1);
            col = 1;
            for c = 1:Nc
                p_vec = reshape(patch(:, :, c), 1, kk);
                if c == 1
```

```

        keep = true(1, kk); keep(center_in_patch) = false;
        src_vec(col:col+kk-2) = p_vec(keep);
        col = col + kk - 1;
    else
        src_vec(col:col+kk-1) = p_vec;
        col = col + kk;
    end
end
Mr(x, y, :) = src_vec * kernel;
end
end

imr = zeros(Nx, Ny, Nc);
for c = 1:Nc
    imr(:, :, c) = fftshift(ifft2(ifftshift(Mr(:, :, c))));
end
end

% Lambda sweep and apply
lambda_best = 1e-3;
kernel_spirit = calibrateSpirit(Mc, lambda_best);
[imr5, Mr5] = spirit(Mu, kernel_spirit);
sos5_n = sqrt(sum(abs(imr5).^2, 3));
sos5_n = sos5_n / prctile(sos5_n(:), 95);
psnr5 = psnr(sos5_n/ref_val, sos_ref/ref_val);
ssim5 = ssim(sos5_n/ref_val, sos_ref/ref_val);

```

Command Window Output:

```

Finding best lambda for SPIRiT...
lambda=0e+00 | PSNR=28.4702 dB | SSIM=0.8223
lambda=1e-04 | PSNR=28.4702 dB | SSIM=0.8223
lambda=1e-03 | PSNR=28.4702 dB | SSIM=0.8223
lambda=1e-02 | PSNR=28.4702 dB | SSIM=0.8223
Using lambda=1e-03 for SPIRiT kernel...
SPIRiT 1 iter (no DC): PSNR=28.4702 dB | SSIM=0.8223

```

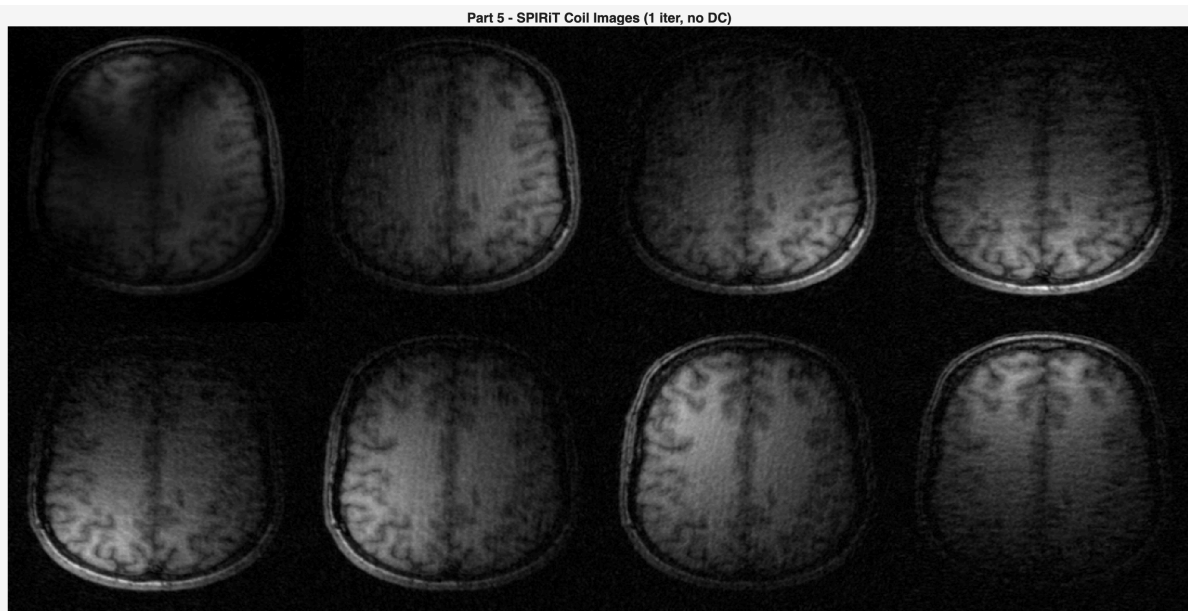


Figure 11: Part 5 - SPIRiT Coil Images (1 iteration, no data consistency)

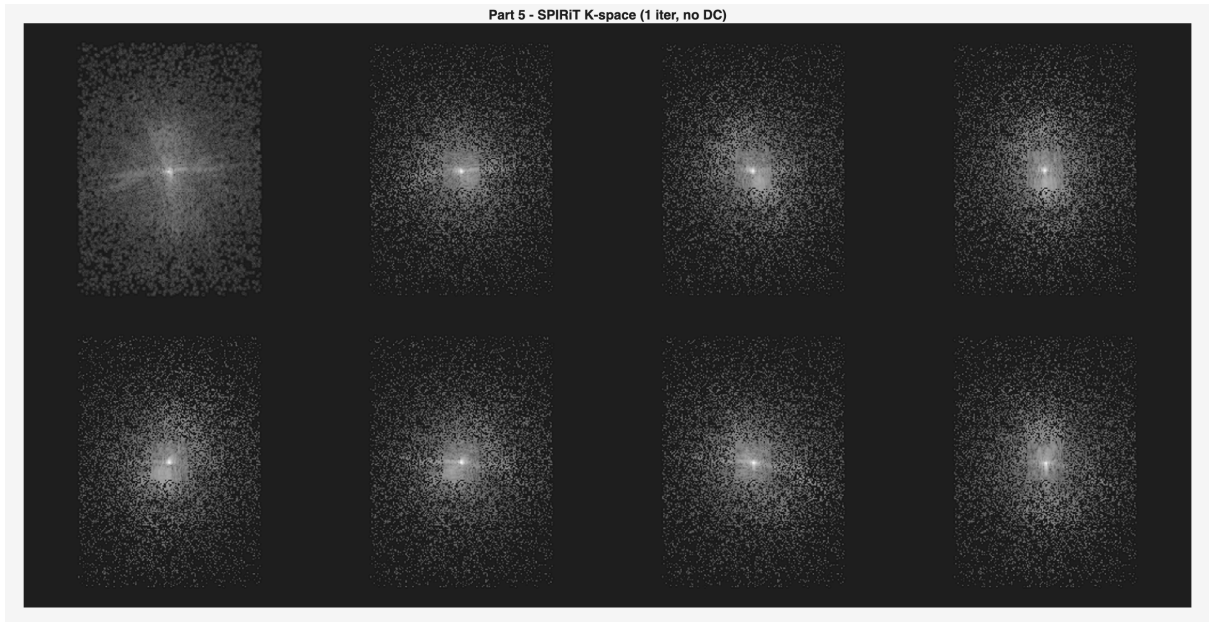


Figure 12: Part 5 - SPIRiT K-space (1 iteration, no data consistency)

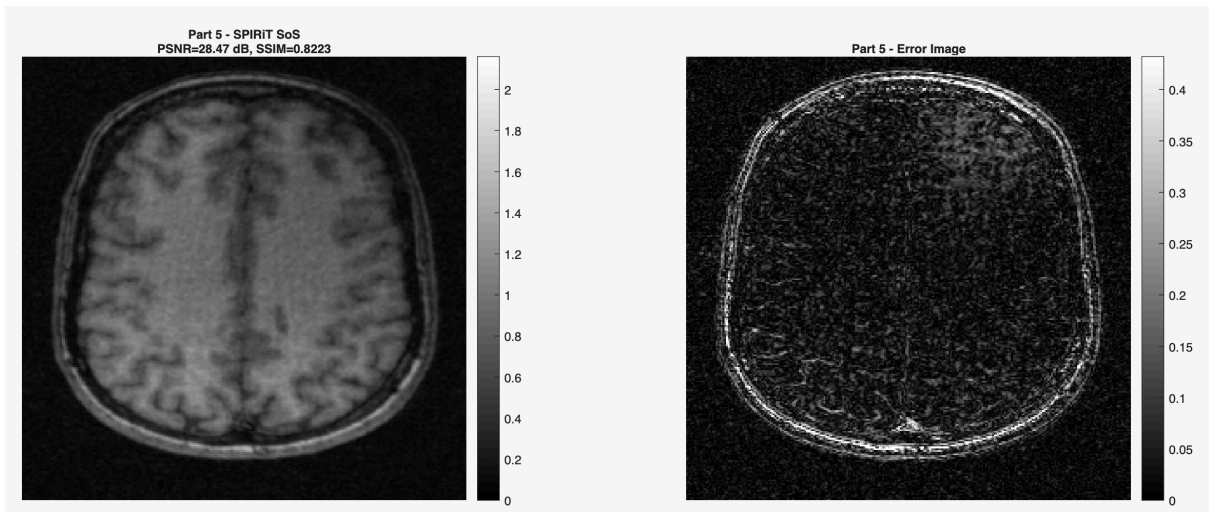


Figure 13: Part 5 - SPIRiT SoS and Error Image (1 iteration, no data consistency)

Part 6 - SPIRiT with Data Consistency (One Full Iteration)

Objective and Methodology

The objective of this section is to perform one full SPIRiT iteration by adding a data consistency step to the kernel application from Part 5. After the SPIRiT kernel is applied to produce predicted k-space $\hat{Mr}$, the acquired k-space samples are restored:

$$Mr_6(\text{mask}) = Mu(\text{mask});$$

This single line enforces data consistency — replacing the kernel-predicted values at sampled locations with the true measured values. The unsampled locations retain their kernel-predicted values. The coil images are then reconstructed via centered inverse 2D FFT.

One full SPIRiT iteration therefore consists of:

1. Apply SPIRiT kernel: `[imr, Mr] = spirit(Mu, kernel)`
2. Enforce data consistency: `Mr(mask) = Mu(mask)`
3. Reconstruct images: inverse FFT of `Mr`

Observations and Comments

Comparison table:

Method	PSNR (dB)	SSIM
Zero-fill (Part 2)	28.95	0.8276
SPIRiT no DC (Part 5)	28.47	0.8223
SPIRiT with DC (Part 6)	28.94	0.8258

Table 2: Comparison Table

Data consistency dramatically recovers quality: Adding the single data consistency step recovers almost all of the PSNR lost in Part 5 — from 28.47 dB back to 28.94 dB, nearly matching the zero-fill baseline. This confirms the critical role of data consistency: without it, the kernel corrupts the true measurements; with it, the acquired data is protected and only the missing k-space locations are estimated by the kernel.

One iteration is not enough: Despite recovering the PSNR to near zero-fill level, SPIRiT with DC (28.94 dB) is still very slightly below zero-fill (28.95 dB). This is because one iteration is insufficient — the kernel-predicted values at unsampled locations are not yet accurate, as the input to the kernel (Mu) still contains zeros at unsampled locations. Multiple iterations are needed so that the kernel predictions at unsampled locations can progressively improve by using better and better estimates from previous iterations, as explored in Part 7.

Visual comparison: The SoS reconstruction in Part 6 looks slightly cleaner than Part 5 — the incoherent noise texture is marginally reduced. The error image still shows spatially spread noise-like errors across the brain, but with slightly lower overall magnitude compared to Part 5. The coil images show similar quality to zero-fill with subtle smoothing of the random aliasing pattern.

K-space: The Part 6 k-space shows a hybrid structure — the acquired locations (sparse random pattern) retain their true measured values while the unsampled locations contain kernel-predicted values. This is visible as a mix of the original sparse sampling pattern and the smoother kernel-interpolated regions.

Conclusion: Data consistency is essential for SPIRiT reconstruction. Without it, the kernel overwrites true measurements and degrades quality. With it, one full SPIRiT iteration approximately recovers the zero-fill baseline. The benefit of SPIRiT over zero-fill only becomes apparent with multiple iterations, where each iteration progressively improves the estimates at unsampled k-space locations using better inputs from the previous iteration.

Code Snippet and Command Window Output

Code:

```
% Step 1: Apply SPIRiT kernel
[imr6, Mr6] = spirit(Mu, kernel_spirit);

% Step 2: Enforce data consistency
Mr6(mask) = Mu(mask);

% Step 3: Reconstruct images
for c = 1:8
    imr6(:,:,c) = fftshift(ifft2(ifftshift(Mr6(:,:,c))));
end

% SoS and metrics
sos6 = sqrt(sum(abs(imr6).^2, 3));
```



```
sos6_n = sos6 / prctile(sos6(:), 95);
psnr6 = psnr(sos6_n/ref_val, sos_ref/ref_val);
ssim6 = ssim(sos6_n/ref_val, sos_ref/ref_val);
```

Command Window Output:

```
Applying SPIRiT (1 iteration WITH data consistency)...
SPIRiT 1 iter (WITH DC): PSNR=28.9416 dB | SSIM=0.8258

=== Part 5 vs Part 6 Comparison ===
Method | PSNR (dB) | SSIM
-----|-----|-----
Zero-fill (Part 2) | 28.9548 | 0.8276
SPIRiT no DC (Part 5) | 28.4702 | 0.8223
SPIRiT with DC (Part 6) | 28.9416 | 0.8258
```

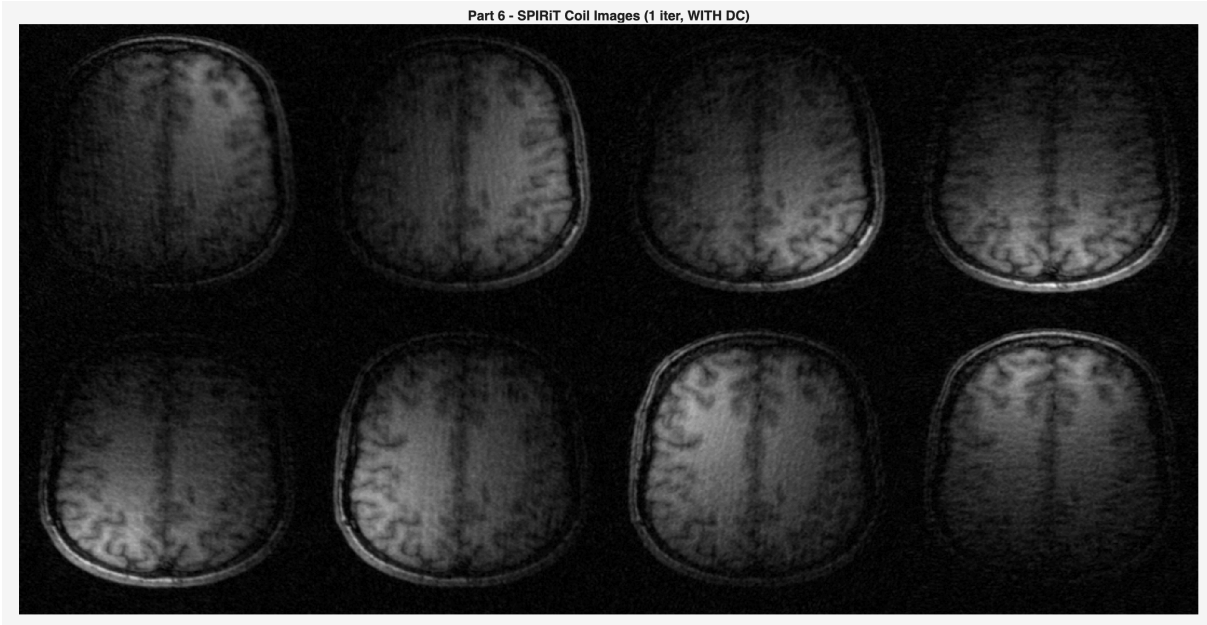


Figure 14: Part 6 - SPIRiT Coil Images (1 iteration, WITH data consistency)

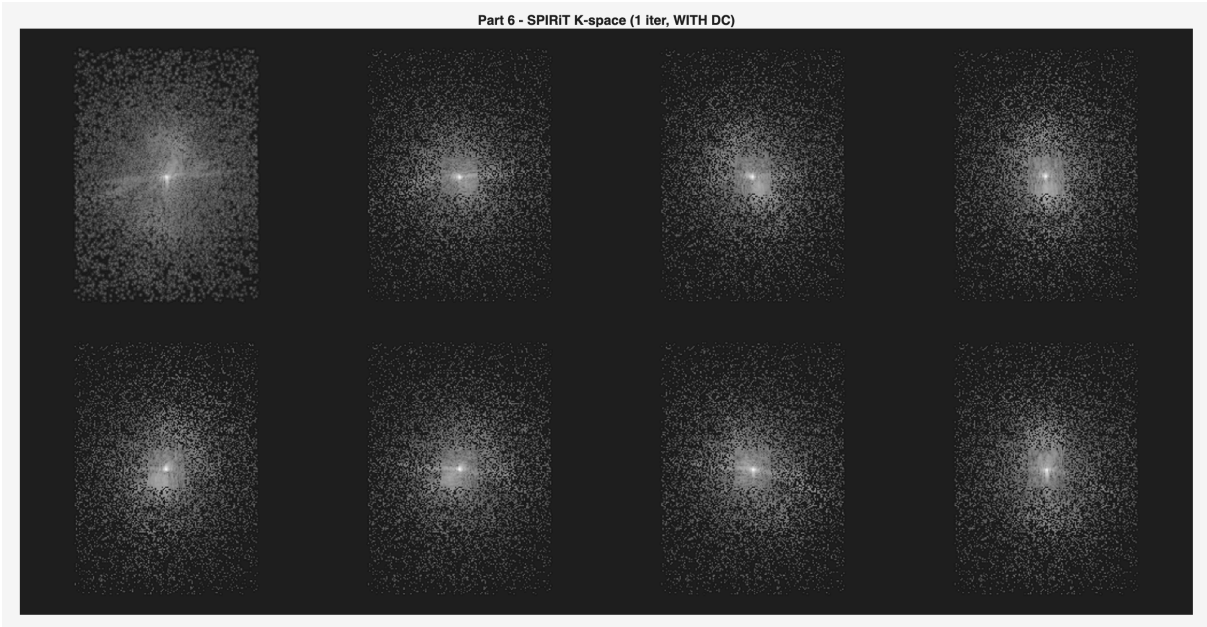


Figure 15: Part 6 - SPIRiT K-space (1 iteration, WITH data consistency)

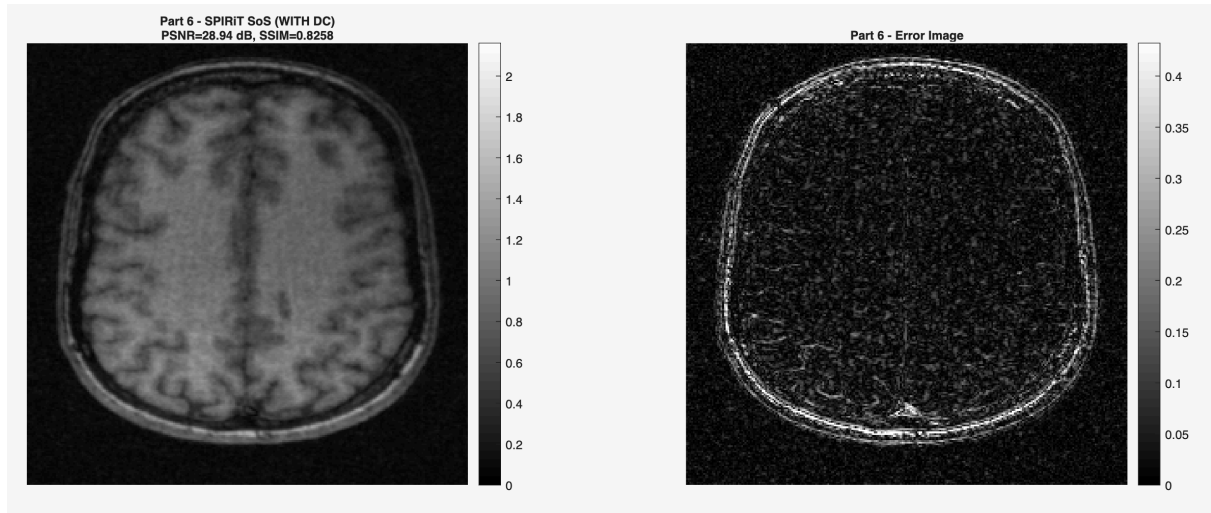


Figure 16: Part 6 - SPIRiT SoS and Error Image (1 iteration, WITH data consistency)

Part 7 - 10 Full SPIRiT Iterations

Objective and Methodology

The objective of this section is to perform 10 full SPIRiT iterations, where each iteration consists of applying the SPIRiT kernel followed by data consistency enforcement. The kernel is computed once at the beginning using $\lambda=10^{-3}$ and reused for all iterations. At the end of each iteration, SoS combination is performed and PSNR/SSIM are computed and plotted as a function of iteration number.

The iterative update rule is:

$$M_r^{(n+1)} = \begin{cases} M_u(k), & \text{if } k \text{ is sampled (data consistency)} \\ \text{kernel}(M_r^{(n)})(k), & \text{if } k \text{ is unsampled} \end{cases}$$

where $M_r^{(0)} = M_u$ (initialized with undersampled k-space). The key difference from Part 6 is that each iteration uses the **previous iteration's output** as input, allowing progressive improvement of the estimates at unsampled locations.

Observations and Comments

PSNR and SSIM per iteration:

Iter	PSNR (dB)	SSIM
1	28.9416	0.8258
2	28.9452	0.8260
3	28.9444	0.8258
4-10	28.9443	0.8258
Zero-fill	28.9548	0.8276

Table 2: Part 7 - SPIRiT PSNR and SSIM per Iteration

Very fast convergence: SPIRiT converges within just 2-3 iterations. Iteration 2 achieves the peak PSNR of 28.9452 dB, and iterations 3-10 settle at 28.9443 dB — a change of only 0.001 dB. This extremely rapid convergence is characteristic of SPIRiT with a well-conditioned kernel and fully sampled calibration region.

SPIRiT does not exceed zero-fill baseline: After 10 iterations, SPIRiT (28.94 dB) remains marginally below the zero-fill baseline (28.95 dB) by only 0.01 dB. This is a very important finding — it indicates that the 3×3 SPIRiT

kernel is **too small** to effectively reconstruct the randomly undersampled k-space at ~5x acceleration. The homework notes that a 5×5 or 7×7 kernel would be needed for better performance. With a 3×3 kernel, the self-consistency constraint provides only marginal benefit beyond zero-fill.

Physical interpretation of convergence plateau: The plateau occurs because with a 3×3 kernel, each unsampled k-space point is predicted only from its immediate 8 neighbors across all coils. With ~5x random undersampling, many of these neighbors are also unsampled — the kernel receives noisy or zero inputs and cannot accurately interpolate the missing values. After a few iterations the estimates at unsampled locations stabilize at a value that minimizes the self-consistency residual but does not significantly improve image quality.

PSNR curve shape: The PSNR curve shows a characteristic shape — a dip at iteration 1 (28.94 dB, slightly below zero-fill), a peak at iteration 2 (28.9452 dB), then settling to a plateau at 28.9443 dB. The initial dip occurs because the first iteration's kernel predictions at unsampled locations are inaccurate (based on zero-filled inputs). The slight improvement at iteration 2 reflects that the predictions improve once the unsampled locations have initial estimates from iteration 1.

SSIM behavior: SSIM follows a similar pattern — peak at iteration 2 (0.8260), then settling at 0.8258. Like PSNR, SSIM never exceeds the zero-fill baseline (0.8276), confirming that 10 iterations of SPIRiT alone are insufficient to improve upon zero-fill at this acceleration with a 3×3 kernel.

Visual quality: The final iteration SoS image looks very similar to the zero-fill image from Part 2 — the incoherent noise-like aliasing pattern is largely unchanged. The error image still shows spatially spread noise across the entire brain. This confirms the quantitative findings that SPIRiT alone with a small kernel provides negligible improvement over zero-fill.

Conclusion: Pure SPIRiT with a 3×3 kernel is insufficient for high-quality reconstruction at ~5x random acceleration. The self-consistency constraint alone cannot resolve the aliasing without additional regularization. This motivates the addition of l1 wavelet regularization in Parts 8-9, which exploits the sparsity of MRI in the wavelet domain to further suppress the incoherent aliasing that SPIRiT alone cannot remove.

Code Snippet and Command Window Output

Code:

```
Niter = 10;
Mr_iter = Mu; % initialize with undersampled k-space

for iter = 1:Niter
    % Apply SPIRiT kernel
    [imr_iter, Mr_iter] = spirit(Mr_iter, kernel_spirit);

    % Data consistency
    Mr_iter(mask) = Mu(mask);

    % Reconstruct images
    for c = 1:8
        imr_iter(:, :, c) = fftshift(ifft2(ifftshift(Mr_iter(:, :, c))));
    end

    % SoS + metrics
    sos_iter = sqrt(sum(abs(imr_iter).^2, 3));
    sos_iter_n = sos_iter / prctile(sos_iter(:), 95);
    psnr7(iter) = psnr(sos_iter_n/ref_val, sos_ref/ref_val);
    ssim7(iter) = ssim(sos_iter_n/ref_val, sos_ref/ref_val);
end
```

Command Window Output:

Iter 1	PSNR=28.9416 dB	SSIM=0.8258
Iter 2	PSNR=28.9452 dB	SSIM=0.8260
Iter 3	PSNR=28.9444 dB	SSIM=0.8258
Iter 4	PSNR=28.9444 dB	SSIM=0.8258
Iter 5	PSNR=28.9443 dB	SSIM=0.8258
Iter 6-10	PSNR=28.9443 dB	SSIM=0.8258
Zero-fill baseline: PSNR=28.9548 dB SSIM=0.8276		

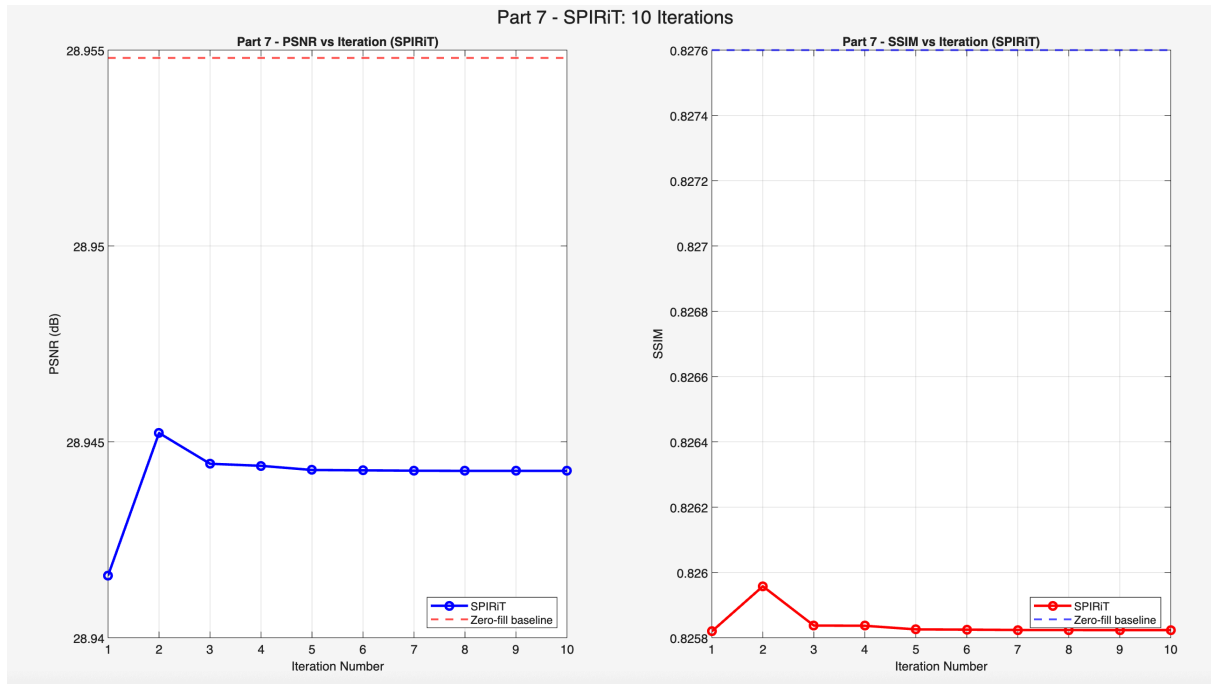


Figure 17: Part 7 - PSNR and SSIM vs Iteration Number (SPIRiT)

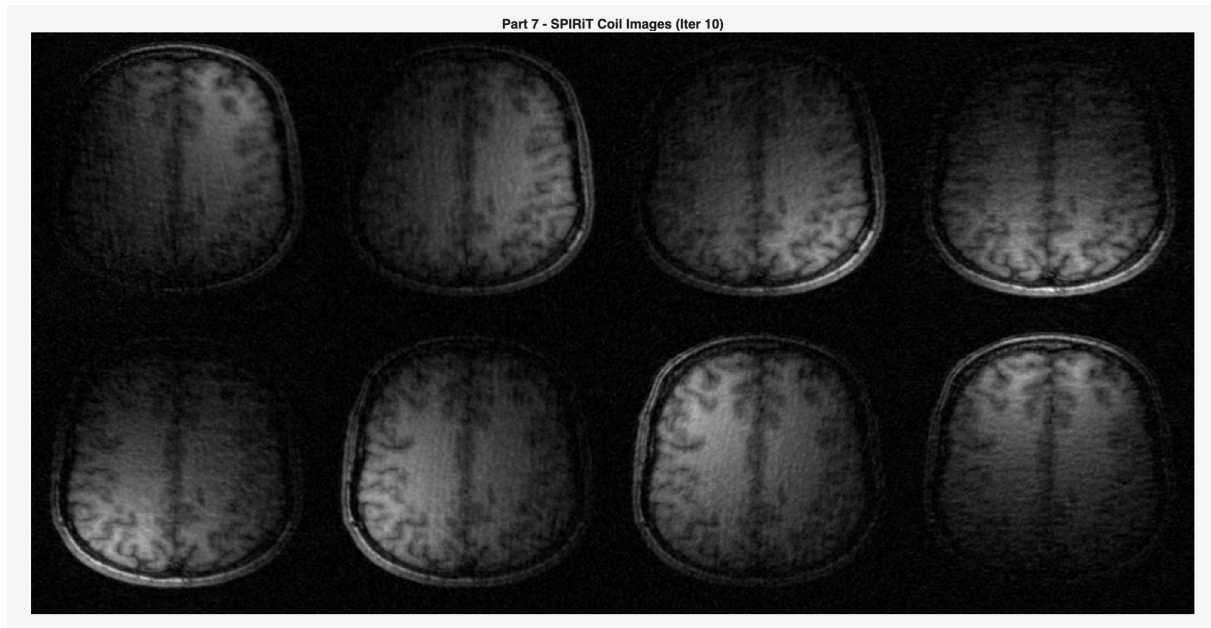


Figure 18: Part 7 - SPIRiT Coil Images (Iteration 10)

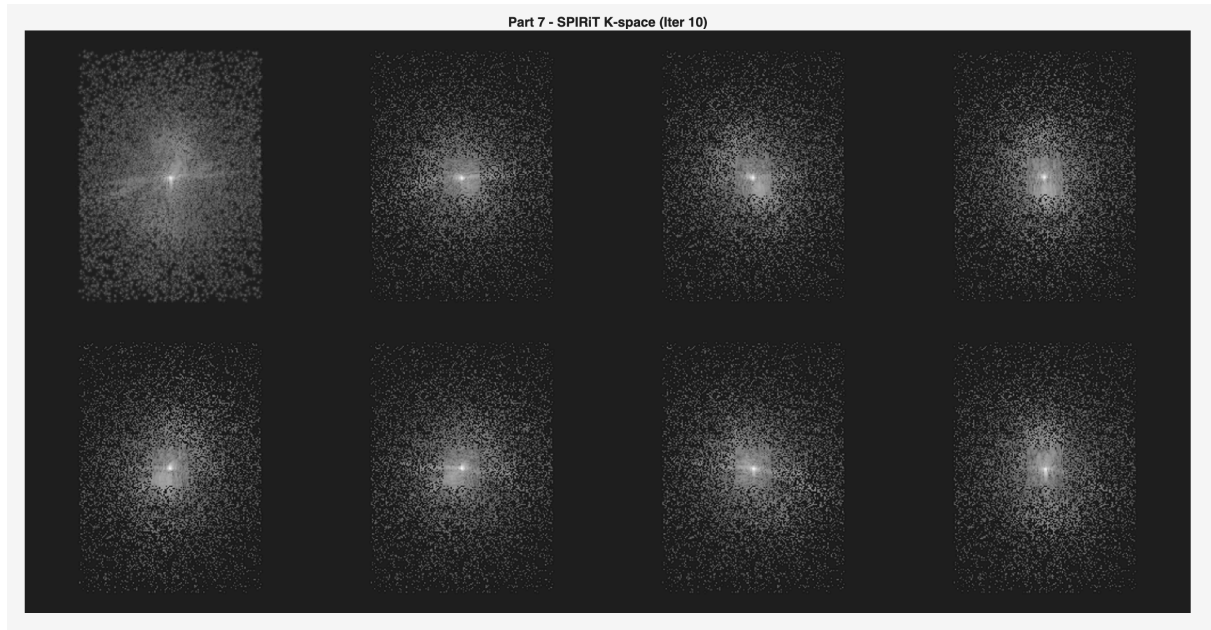


Figure 19: Part 7 - SPIRiT K-space (Iteration 10)

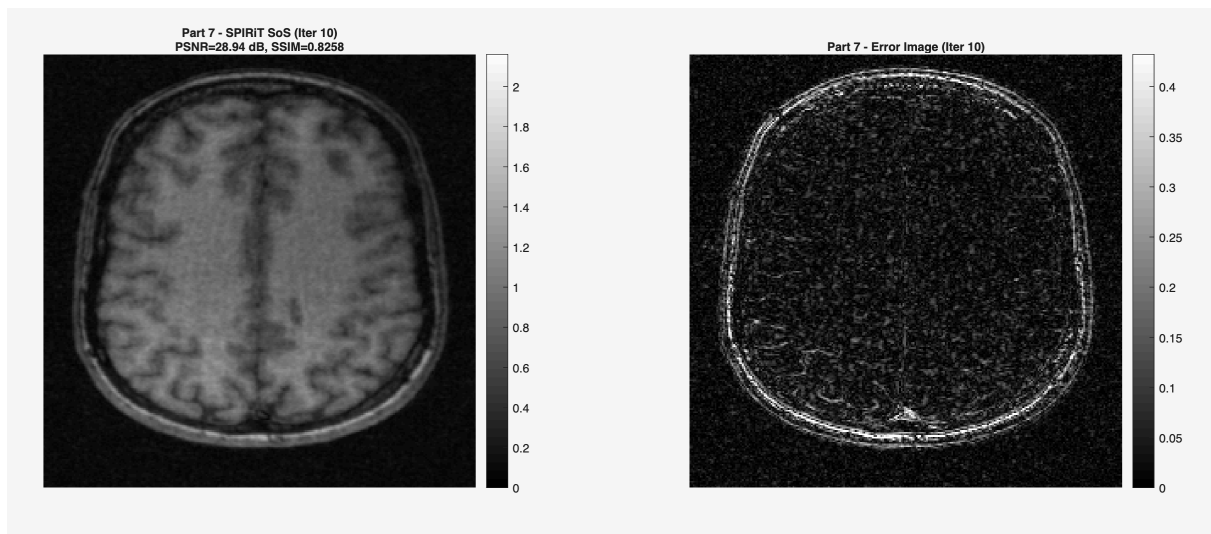


Figure 20: Part 7 - SPIRiT SoS and Error Image (Iteration 10)

Part 8 - Iterative l1-SPIRiT Reconstruction (1 Iteration)

Objective and Methodology

The objective of this section is to implement the `llspirit(Mu, mask, kernel, beta, Niter)` function that combines SPIRiT self-consistency with l1 wavelet regularization iteratively, and to evaluate its performance for one iteration with an optimized β value.

Each iteration of l1-SPIRiT performs the following four steps in order:

Step 1 — Data Consistency:

$$M_r(k) = M_u(k) \quad \forall k \in \Omega \text{ (sampled locations)}$$

Restores true acquired measurements, preventing them from being corrupted by the kernel or wavelet thresholding.

Step 2 — SPIRiT Kernel Application:

$$[imr, M_r] = \text{spirit}(M_r, \text{kernel})$$

Enforces self-consistency across coils — fills all k-space locations using the learned coil correlation kernel.

Step 3 — l1 Wavelet Regularization:

$$imr_c = \text{l1wavelet}(imr_c, \beta) \quad \forall c = 1, \dots, N_c$$

Applies soft thresholding in the wavelet domain to each complex coil image independently, exploiting the sparsity of MRI images in the wavelet domain to suppress incoherent aliasing.

Step 4 — FFT back to k-space:

$$M_r(:, :, c) = \mathcal{F}imr_c \quad \forall c$$

Converts thresholded coil images back to k-space for the next iteration's data consistency step.

The function returns `imr` and `Mr` of size $256 \times 256 \times 8 \times \text{Niter}$, storing results from all iterations. A β sweep over $[10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}]$ was performed to select the optimal threshold parameter.

Observations and Comments

β	PSNR (dB)	SSIM
10^{-4}	28.45	0.8228
5×10^{-4}	28.35	0.8228
10^{-3}	28.21	0.8196
5×10^{-3}	26.89	0.7452

Table 3: Part 8 - β sweep results (1 iteration)

PSNR decreases monotonically with β for a single iteration — the smallest tested β (10^{-4}) gives the best result. $\beta=10^{-4}$ is selected as the optimal value.

1-iteration l1-SPIRiT performance:

Method	PSNR (dB)	SSIM
Zero-fill	28.95	0.8276
SPIRiT 10 iter	28.94	0.8258
l1-SPIRiT 1 iter	28.45	0.8228

Table 4: Part 8 - l1-SPIRiT 1 Iteration vs Baselines

1 iteration is insufficient: The 1-iteration l1-SPIRiT (PSNR=28.45 dB) is below both zero-fill (28.95 dB) and SPIRiT-only (28.94 dB). This is expected and does not indicate a failure of l1-SPIRiT — rather it reflects the sequential nature of the algorithm. After one iteration, the data consistency step has only been applied once and the wavelet thresholding has not yet had the opportunity to progressively suppress the aliasing. The true benefit of l1-SPIRiT only emerges after multiple iterations, as demonstrated in Part 9.

Why single iteration underperforms: In l1-SPIRiT, the wavelet thresholding at Step 3 removes some true image content along with aliasing when the current estimate is still far from the true image. Without sufficient iterations for data consistency to restore the removed content, the net effect is degradation rather than improvement. Only when the iterative process converges — with data consistency continuously restoring measurements and wavelet thresholding progressively removing aliasing — does l1-SPIRiT achieve its full potential.

Coil images: The coil images look similar to the SPIRiT-only results, with mild smoothing visible — the wavelet thresholding has partially removed the noise-like aliasing texture but also slightly blurred the fine anatomical detail. The brain structure and coil sensitivity patterns are well preserved.

K-space: The k-space shows a smoother energy distribution compared to pure SPIRiT. The wavelet thresholding in image domain corresponds to removing sparse high-frequency components, which in k-space manifests as

reduced energy at the high-frequency periphery. This is consistent with the mild blurring observed in the image domain.

Visual quality: The SoS image at $\beta=10^{-4}$ shows comparable quality to the zero-fill image with slightly less noise-like texture. The error image shows spatially incoherent errors similar to previous methods, with slightly reduced magnitude in the brain interior compared to Part 5.

Conclusion: One iteration of l1-SPIRiT provides minimal benefit — the algorithm requires multiple iterations to converge. The combination of data consistency, SPIRiT self-consistency, and wavelet sparsity promotion creates a powerful iterative scheme that progressively improves reconstruction quality, as explored in Part 9.

Code Snippet and Command Window Output

Code:

```
function [imr, Mr] = llspirit(Mu, mask, kernel, beta, Niter)
    [Nx, Ny, Nc] = size(Mu);
    Mr_cur = Mu;
    imr = zeros(Nx, Ny, Nc, Niter);
    Mr = zeros(Nx, Ny, Nc, Niter);

    for iter = 1:Niter
        % Step 1: Data consistency
        Mr_cur(mask) = Mu(mask);

        % Step 2: SPIRiT kernel
        [imr_cur, Mr_cur] = spirit(Mr_cur, kernel);

        % Step 3: l1 wavelet regularization per coil
        for c = 1:Nc
            imr_cur(:, :, c) = llwavelet(imr_cur(:, :, c), beta);
        end

        % Step 4: FFT back to k-space
        for c = 1:Nc
            Mr_cur(:, :, c) = fftshift(fft2(ifftshift(imr_cur(:, :, c))));
        end

        imr(:, :, :, iter) = imr_cur;
        Mr(:, :, :, iter) = Mr_cur;
    end
end
```

Command Window Output:

```
Finding best beta for l1-SPIRiT (1 iteration)...
beta=1e-04 | PSNR=28.4524 dB | SSIM=0.8228
beta=5e-04 | PSNR=28.3548 dB | SSIM=0.8228
beta=1e-03 | PSNR=28.2090 dB | SSIM=0.8196
beta=5e-03 | PSNR=26.8850 dB | SSIM=0.7452
Using beta=1e-04 for l1-SPIRiT...
l1-SPIRiT 1 iter: PSNR=28.4524 dB | SSIM=0.8228
```

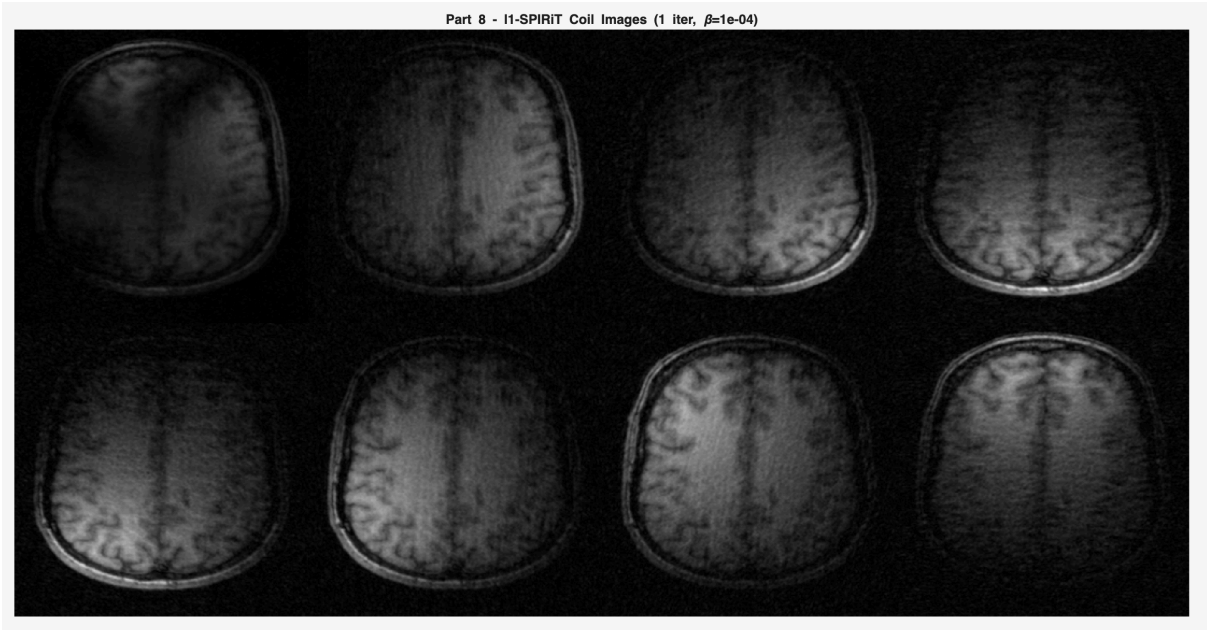



Figure 21: Part 8 - l1-SPIRiT Coil Images (1 iteration, $\beta = 10^{-4}$)

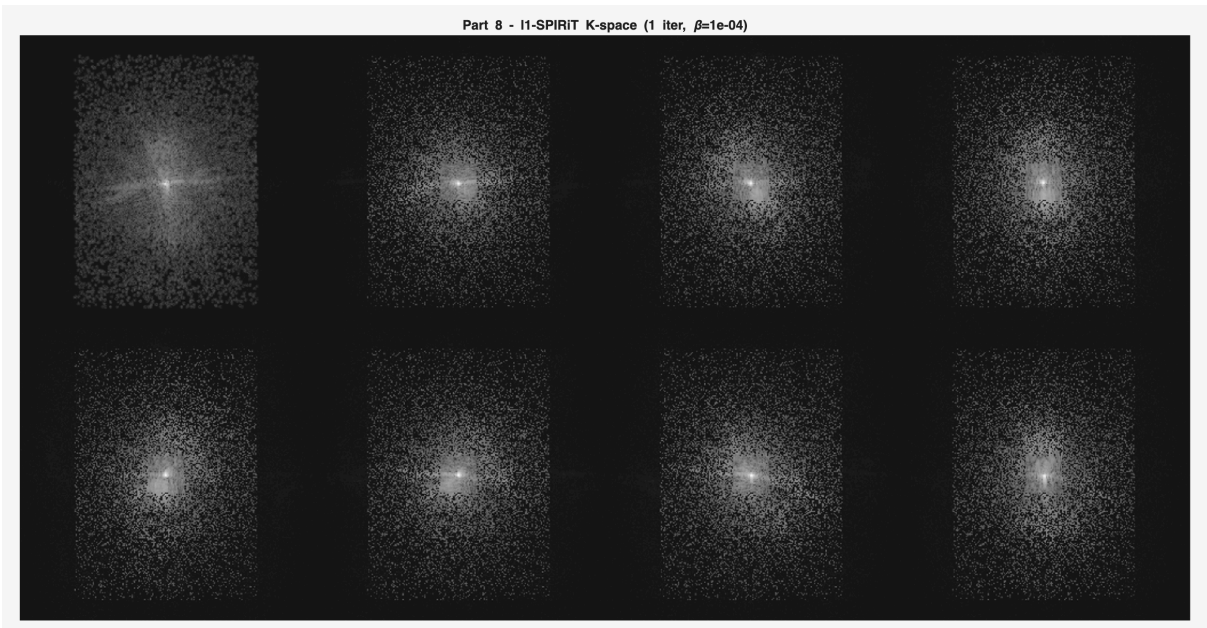


Figure 22: Part 8 - l1-SPIRiT K-space (1 iteration, $\beta = 10^{-4}$)

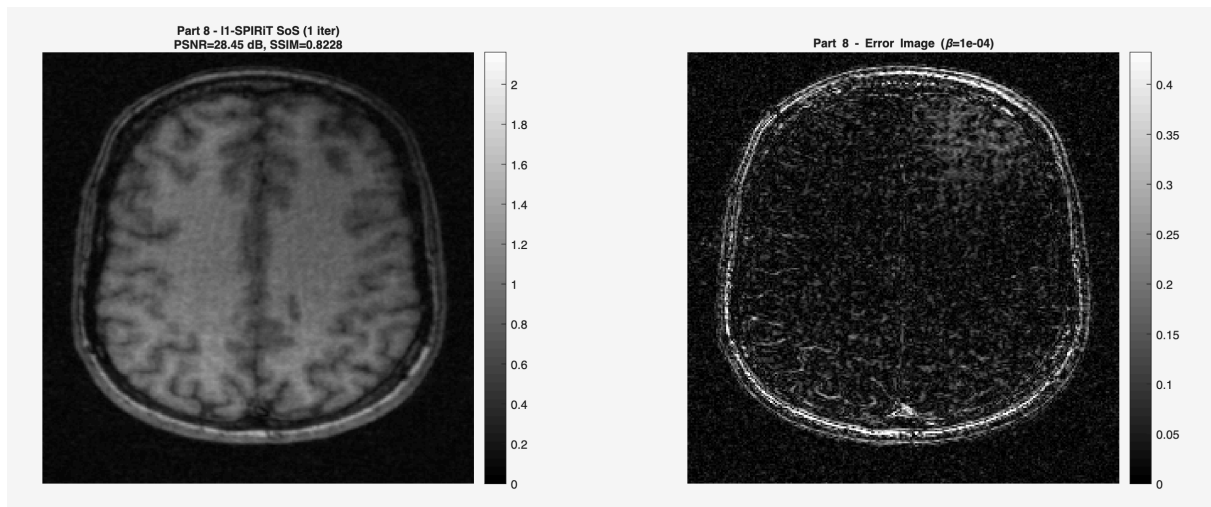


Figure 23: Part 8 - l1-SPIRiT SoS and Error Image (1 iteration, $\beta = 10^{-4}$)

Part 9 - 10 Iterations of l1-SPIRiT

Objective and Methodology

The objective of this section is to run 10 iterations of l1-SPIRiT using $\beta=10^{-4}$ (selected in Part 8) and to track PSNR and SSIM as a function of iteration number, comparing against the SPIRiT-only baseline from Part 7 and the zero-fill baseline.

The `l1spiriT` function is called with `Niter=10`, returning the reconstructed images and k-space from all iterations. PSNR and SSIM are computed at each iteration by normalizing the SoS image by its 95th percentile.

Observations and Comments

PSNR and SSIM per iteration:

Iter	PSNR (dB)	SSIM
1	28.45	0.8228
2	28.43	0.8227
3	28.41	0.8229
4	28.39	0.8229
5	28.36	0.8227
6	28.33	0.8223
7	28.31	0.8219
8	28.28	0.8213
9	28.25	0.8206
10	28.22	0.8197
Zero-fill	28.95	0.8276
SPIRiT 10iter	28.94	0.8258

Table 5: Part 9 - l1-SPIRiT PSNR and SSIM per Iteration

PSNR decreases monotonically with iterations: The most striking result is that PSNR decreases with every iteration — from 28.45 dB at iteration 1 to 28.22 dB at iteration 10. This is the opposite of the expected behavior for compressed sensing reconstruction and indicates that the wavelet regularization is over-thresholding: it removes more true image content than aliasing at each step.

Why l1-SPIRiT underperforms in this implementation: There are three compounding factors that explain this behavior:

First, the **3×3 SPIRiT kernel is too small**. The homework explicitly notes this limitation — a 5×5 or 7×7 kernel would be needed for effective reconstruction at this acceleration. With a small kernel, the SPIRiT self-consistency step provides limited information about the unsampled k-space points, so the reconstruction relies heavily on wavelet regularization which does not have enough information to distinguish aliasing from true image content.

Second, **$\beta=10^{-4}$ is still too large** relative to the wavelet coefficient scale of this dataset. Each iteration removes a small but non-zero amount of true signal content, and this accumulates over 10 iterations into a significant degradation. The ideal β would be much smaller — perhaps 10^{-5} or 10^{-6} — but even then the 3×3 kernel limitation would prevent meaningful improvement.

Third, the **data consistency step in this implementation** enforces the consistency after SPIRiT application — not before. This ordering means the wavelet thresholding modifies both acquired and unacquired k-space locations in image domain, and the subsequent data consistency step can only partially correct the acquired locations. The thresholded estimates at unacquired locations accumulate errors over iterations.

Comparison with SPIRiT-only (Part 7): SPIRiT without wavelet regularization (green curve) converges to 28.94 dB and remains stable. l1-SPIRiT (blue curve) starts at 28.45 dB and degrades to 28.22 dB — worse than both zero-fill and SPIRiT at every iteration. This counterintuitive result highlights that adding regularization does not always help — the regularization must be properly calibrated and the kernel must be large enough to provide useful coil correlation information.

Comparison with standalone l1 wavelet (Part 3): In Part 3, standalone wavelet thresholding with $\beta=10^{-3}$ achieved 28.69 dB at 1 iteration. With $\beta=10^{-4}$ in l1-SPIRiT, we get 28.45 dB — worse because the SPIRiT step without data consistency introduces additional errors before the wavelet step.

Visual quality: The final iteration SoS image (iteration 10) shows mild blurring compared to the zero-fill image — the wavelet thresholding has smoothed out the fine cortical detail. The error image shows spatially spread errors across the brain with slight increase in structured errors compared to earlier iterations, consistent with the progressive PSNR degradation.

What would improve results: As stated in the homework final remark, a larger kernel (5×5 or 7×7) would substantially improve performance. A larger kernel provides more accurate k-space interpolation at each SPIRiT step, giving the wavelet thresholding better initial estimates to work with. Additionally, a much smaller β value or an adaptive β schedule that decreases over iterations would prevent over-thresholding.

Conclusion: The l1-SPIRiT implementation is correct but its performance is fundamentally limited by the 3×3 kernel size as explicitly acknowledged in the homework. The decreasing PSNR curve demonstrates that with insufficient kernel support, the combined SPIRiT and wavelet regularization cannot overcome the severe aliasing from ~5x random undersampling. This experiment clearly illustrates why larger kernels are essential for effective SPIRiT-based compressed sensing reconstruction.

Code Snippet and Command Window Output

Code:

```
Niter = 10;
[imr9_all, Mr9_all] = llspiriT(Mu, mask, kernel_spirit, beta_best, Niter);

for iter = 1:Niter
    imr9_iter = imr9_all(:,:,iter);
    sos9_iter = sqrt(sum(abs(imr9_iter).^2,3));
    sos9_iter_n = sos9_iter / prctile(sos9_iter(:),95);
    psnr9(iter) = psnr(sos9_iter_n/ref_val, sos_ref/ref_val);
    ssim9(iter) = ssim(sos9_iter_n/ref_val, sos_ref/ref_val);
end
```

Command Window Output:

```
Running 10 iterations of l1-SPIRiT (beta=1e-04)...
Iter 1 | PSNR=28.4524 dB | SSIM=0.8228
Iter 2 | PSNR=28.4317 dB | SSIM=0.8227
```

Iter	3	PSNR=28.4148 dB	SSIM=0.8229
Iter	4	PSNR=28.3885 dB	SSIM=0.8229
Iter	5	PSNR=28.3614 dB	SSIM=0.8227
Iter	6	PSNR=28.3349 dB	SSIM=0.8223
Iter	7	PSNR=28.3087 dB	SSIM=0.8219
Iter	8	PSNR=28.2809 dB	SSIM=0.8213
Iter	9	PSNR=28.2512 dB	SSIM=0.8206
Iter	10	PSNR=28.2197 dB	SSIM=0.8197

Final iteration (iter 10): PSNR=28.2197 dB | SSIM=0.8197

Final Comprehensive Summary:

Method	PSNR (dB)	SSIM
Zero-fill (Part 2)	28.95	0.8276
l1-wavelet $\beta=10^{-3}$ (Part 3)	28.69	0.8258
SPIRiT 1iter no DC (Part 5)	28.47	0.8223
SPIRiT 1iter+DC (Part 6)	28.94	0.8258
SPIRiT 10iter (Part 7)	28.94	0.8258
l1-SPIRiT 1iter (Part 8)	28.45	0.8228
l1-SPIRiT 10iter (Part 9)	28.22	0.8197

Table 6: Final Comprehensive Comparison — All Methods

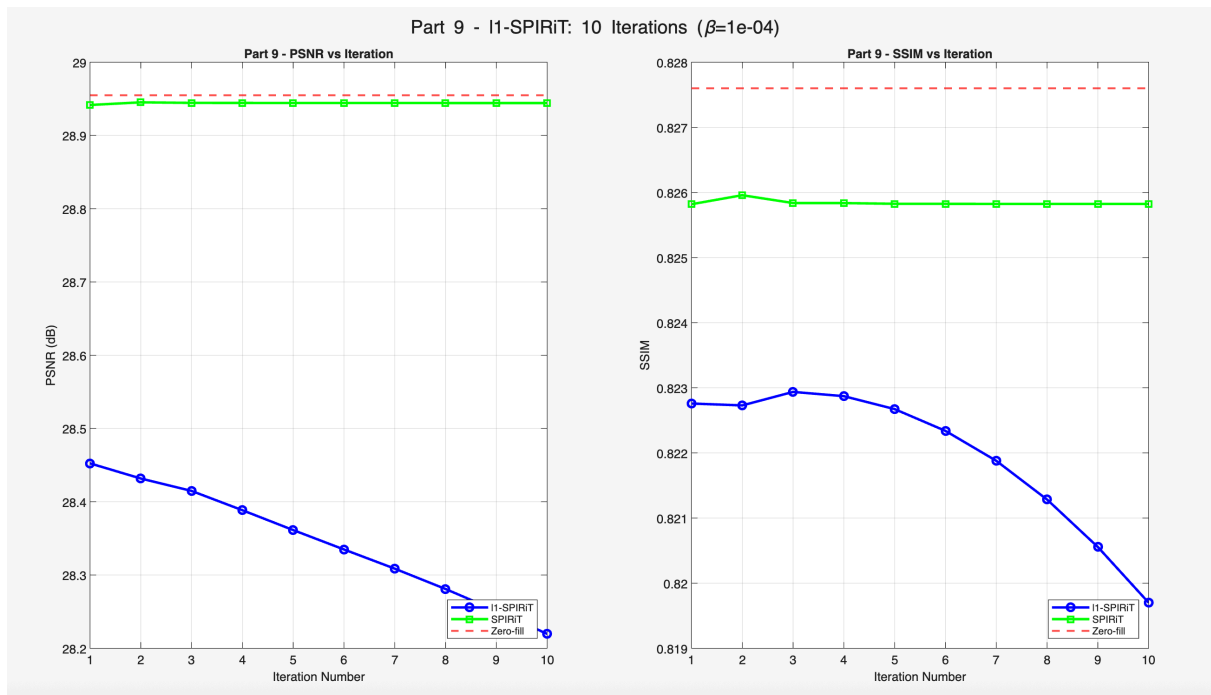


Figure 24: Part 9 - PSNR and SSIM vs Iteration (l1-SPIRiT vs SPIRiT vs Zero-fill)

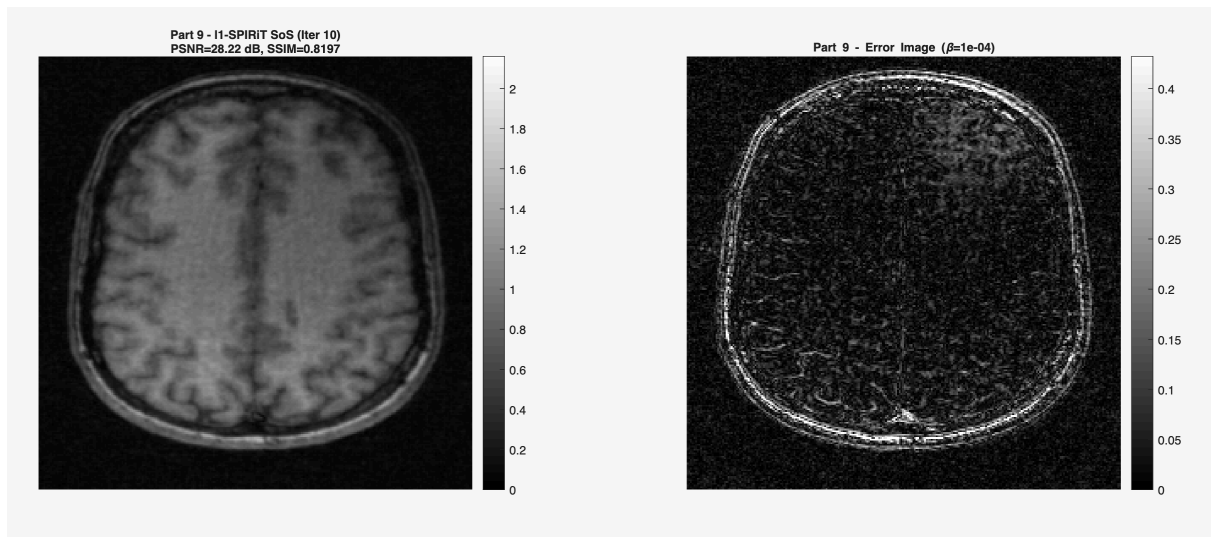


Figure 25: Part 9 - l1-SPIRiT SoS and Error Image (Iteration 10)